

PRESEARCH.md

Complete before writing any code. Save your AI conversation as a reference document. The goal is to make informed decisions about your agent's responsibilities and architecture. You don't have to be right — you have to show your thought process.

Phase 1: Define Your Agent

1. Agent Responsibility Scoping

What events in Ship should the agent monitor proactively?

Five monitoring domains, ordered by signal clarity:

1. Issue lifecycle stalls — triage backlog age, in_progress without updates, in_review duration, overdue past due_date, state regressions (done→reopened)
2. Sprint health — scope creep (issues added mid-sprint via scope-changes endpoint), missing plans/retros, low iteration pass rates, empty sprints
3. Accountability gaps — missing standups, unsubmitted weekly plans/retros, approval bottlenecks (changes_requested with no follow-up)
4. Project drift — stale projects (no activity for N days), unvalidated plans after sprint start, missing RACI ownership, no success criteria defined
5. Team load signals — overloaded assignees (disproportionate in_progress counts), silent team members (no activity for N days)

What constitutes a condition worth surfacing?

The filter: “Would a competent PM, seeing this data right now, change what they do today?” If no, stay quiet. Concretely:

- Connected to an upcoming deadline (sprint end, plan due date)
- Blocking other work or people
- Pattern, not one-off (second missed standup matters more than first)
- Exceeds a grace period (not the instant something goes stale)
- Responsible person hasn't been notified about this condition recently (cooldown)

What can it do autonomously?

- Read any Ship API data
- Compute health scores, risk assessments, trend analysis
- Persist its own findings/state (to Ship DB via new API endpoints we add)
- Generate finding records visible in the UI

What must it always ask a human about?

- Creating or modifying document content
- Changing issue state (moving stale issues, cancelling, etc.)
- Reassigning work
- Any notification to a team member (“I think you should tell Sarah about X” — not sending it directly)

Who does it notify, and under what conditions?

Follow Ship's existing ownership graph:

Scope	Who Gets It
Issue-level	Assignee, then sprint owner if unassigned
Sprint-level	Sprint owner

Scope	Who Gets It
Project-level	Project owner (properties.owner_id)
Program-level	Program owner
Cross-cutting (team load, silent members)	The person's reports_to, or workspace admin

Conditions: severity threshold + grace period expired + not already notified for this condition within cooldown window.

How does it know who is on a project and what their role is?

- GET /api/team/grid — members with project allocations
- Person documents: properties.user_id, properties.role, properties.reports_to
- RACI fields on projects/programs: owner_id, accountable_id, consulted_ids, informed_ids
- Sprint properties.owner_id, issue properties.assignee_id

How does the on-demand mode use context from the current view?

The chat is embedded in Ship's UI. It receives the current document ID + type + active tab. This seeds the Context node:

- On an issue → knows state, assignee, sprint, project, blockers, history
- On a sprint → knows all issues, plan status, team, scope changes, timeline
- On a project → knows all sprints, issues, RACI, ICE scores, velocity
- On a program → knows all projects, cross-project patterns, resource allocation
- On dashboard/my-week → knows the user's full portfolio, what's due, what's stuck

The agent fetches the relevant subgraph from that starting point, not the whole workspace.

2. Use Case Discovery (minimum 5)

Starting from pain points by role, not from features:

Director — “Are my projects actually on track, or just green?” Can't see through status theater. Needs early warning before things blow up. Doesn't have time to dig through every sprint.

PM — Loses track of stuck issues. Sprint scope creeps without noticing. Triage backlog grows silently. Plans/retros pile up, approvals bottleneck. Hard to know which team members need support vs. are fine.

Engineer — Blocked and nobody notices. Finishes a task and isn't sure what to pick up next. Wants to understand how their work connects to the bigger picture.

#	Role	Trigger	Agent Detects / Produces	Human Decides
1	PM	Proactive, mid-sprint	Sprint scope creep — issues added after sprint start, quantified as % of original scope, shows who added what and when	Whether to remove added scope, extend sprint, or accept and re-prioritize
2	PM	Proactive, daily	Stale triage backlog — issues in triage >48hrs, grouped by project/program, with suggested accept/reject based on priority and team capacity	Accept, reject, or reassign each issue

#	Role	Trigger	Agent Detects / Produces	Human Decides
3	Director	Proactive, weekly + on-demand	Accountability debt roll-up — missing plans, retros, standups, unresolved approvals across all teams. Ranked by severity and organizational impact	Whether to escalate, which items to address first
4	PM/Engineer	On-demand, from issue or sprint view	Blocked work chain — traces full dependency chain: what's blocked, what's blocking, who owns each link, how long each link has been stuck	Priority reordering, reassignment, scope cuts
5	Director/PM	On-demand, from project view	Project risk assessment — composite score from velocity trends, issue staleness, missing ownership, plan validation gaps, ICE drift. Explains why it's risky, not just that it is	Whether to intervene, how to reallocate resources
6	Engineer	On-demand, from dashboard	Smart next action — given your assigned issues, sprint priorities, blockers, and team state, recommends what to pick up next with reasoning	Whether to follow the recommendation
7	PM/Director	On-demand, from program view	Retro pattern mining — across multiple sprint retros, surfaces recurring themes: repeated blockers, workarounds becoming permanent, same failure modes across sprints	Which patterns to address structurally vs. accept

UC7 is the differentiator. Most agents do point-in-time health checks. Mining across retros to find patterns that humans miss because they only read one retro at a time — that's the graph reasoning the PRD is asking for.

3. Trigger Model Decision

Choice: Hybrid — WebSocket listener + scheduled poll.

Why: Ship already has a `/events` WebSocket that broadcasts `accountability:updated` on every document create/update, issue state change, plan/retro update, and approval action. That's exactly the event surface FleetGraph needs. We connect as a client and get near-real-time triggers. A lightweight poll every 5 minutes catches anything missed during reconnection gaps.

Why not pure poll: To hit $<5\text{min}$ detection latency with polling alone, you need $\leq 5\text{min}$ intervals. That's 288 polls/day per workspace, most returning "nothing changed." Wasteful and still has up to 5min lag.

Why not webhooks: Ship has no webhook infrastructure. We could build it, but that's significant Ship-side work for MVP when the WebSocket channel already exists and carries the events we need. (Remains an option if WebSocket proves insufficient.)

Why not pure WebSocket: WebSocket connections drop. Network blips, deploys, restarts. The 5-minute poll is a safety net that guarantees no event is missed for longer than 5 minutes regardless of connection state.

Cost at scale:

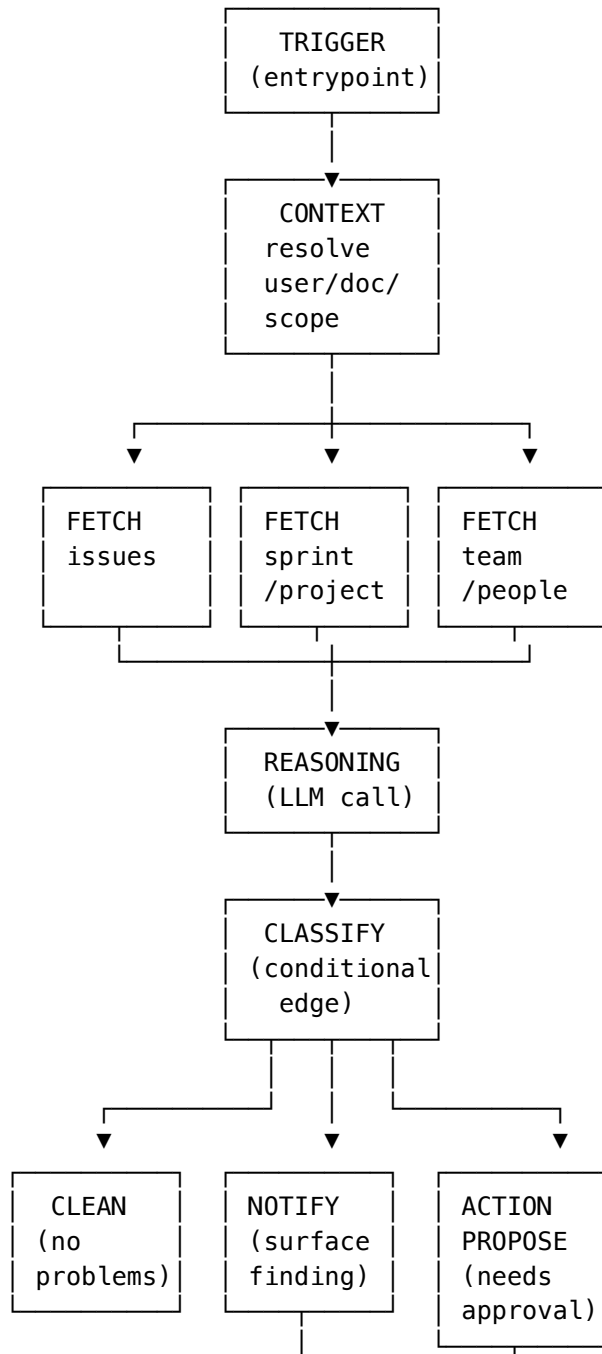
- 100 projects: 1 WebSocket connection per workspace (not per project). Poll hits ~3-4 lightweight endpoints per cycle. Negligible.
- 1,000 projects: Same model. WebSocket connections scale with workspaces, not projects. Poll cost is linear but each call is a timestamp comparison, not a full data fetch.

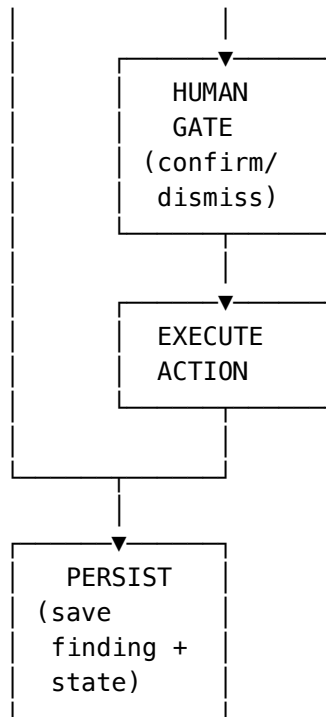
Detection latency: WebSocket events arrive in <1 second. The graph then needs to fetch context and reason — call it 10-30 seconds for the full pipeline. Well under 5 minutes. Poll safety net adds at most 5 minutes for missed events.

Phase 2: Graph Architecture

4. Node Design

Both proactive and on-demand modes share the same graph — the difference is how they enter it.





Node inventory:

Node	Type	Purpose	Parallel?
Trigger	Entry	Routes proactive (WebSocket/poll event) vs. on-demand (user chat). Sets mode flag.	—
Context	Context	Resolves who's asking, what they're looking at, their role, scope of analysis. Proactive: scope = workspace. On-demand: scope = current document + neighbors.	—
Fetch Issues	Fetch	GET /api/issues filtered by scope	Yes, parallel with other fetches
Fetch Sprint/Project	Fetch	GET /api/weeks/:id, /api/projects/:id, associations, scope-changes	Yes, parallel
Fetch Team/People	Fetch	GET /api/team/grid, person docs, accountability items	Yes, parallel

Node	Type	Purpose	Parallel?
Reasoning	Reasoning (LLM)	The core. Takes fetched data, analyzes relationships, gaps, risk. Produces structured findings with severity + affected parties + recommended actions. Not summarizing — reasoning.	—
Classify	Conditional edge	Routes based on reasoning output: clean (no problems) / notify (surface finding, no action needed) / action-propose (consequential action identified)	—
Clean	Output	No problems found. Proactive: silent. On-demand: “Everything looks healthy, here’s why.”	—
Notify	Action	Persists finding, surfaces it in UI. No human gate needed — it’s informational.	—
Action Propose	Action	Proposes a concrete action (reassign issue, change state, escalate). Requires approval.	—
Human Gate	HITL	Pauses graph. Presents proposed action to user with context. Waits for confirm/dismiss/snooze.	—
Execute Action	Action	Carries out the approved action via Ship API (PATCH issue, POST comment, etc.)	—
Persist	Output	Saves finding record + updates FleetGraph state (last check timestamps, notification cooldowns, finding history)	—

Which fetch nodes run in parallel? All three fetch nodes run in parallel — they’re independent API calls. The Reasoning node waits for all three to complete.

Conditional edges and what triggers each branch:

Edge	Condition
Classify → Clean	Reasoning found no conditions worth surfacing
Classify → Notify	Finding detected, informational only (health score, trend, pattern)
Classify → Action Propose	Finding includes a concrete recommended action that would modify Ship state

Edge	Condition
Human Gate → Execute	User confirms proposed action
Human Gate → Persist	User dismisses or snoozes (skip execution, still record the finding)

On-demand adds a loop: After Persist, on-demand mode can return to Reasoning if the user asks a follow-up question. Proactive mode always terminates.

5. State Management

State carried within a single graph run (LangGraph state):

```
{
  mode: "proactive" | "on_demand",
  trigger_event: { type, data, timestamp },

  // Context
  user_id: string | null,          // null for proactive
  document_id: string | null,      // null for proactive
  document_type: string | null,
  scope: "workspace" | "program" | "project" | "sprint" | "issue",

  // Fetched data (populated by fetch nodes)
  issues: Issue[],
  sprints: Sprint[],
  projects: Project[],
  team_members: Person[],
  accountability_items: ActionItem[],

  // Reasoning output
  findings: Finding[],             // structured: severity, affected, recommendation
  proposed_actions: Action[],      // concrete Ship API mutations

  // HITL
  human_decision: "pending" | "confirmed" | "dismissed" | "snoozed" | null,

  // Chat history (on-demand only)
  messages: Message[],
}
```

State persisted between proactive runs — as Ship documents:

FleetGraph state lives in the unified document model. New document types added to the enum:

- **fleetgraph_finding** — each finding is a document. Properties store severity, affected_entity_id, finding_type, status (active/dismissed/snoozed/resolved), proposed_action, human_decision, snooze_until. Content stores the LLM's reasoning narrative. Gets full history tracking, associations (linked to the issue/sprint/ project it's about), and audit trail for free.
- **fleetgraph_config** (singleton per workspace) — stores notification cooldowns, check timestamps, agent preferences. Properties hold the operational state.

Benefits of staying in the unified model:

- Findings are visible to all workspace members — no shadow state

- Existing document infrastructure: history, associations, search, audit logs
- Association table links findings to the entities they concern
- Comments work — team can discuss a finding inline
- Backlinks work — viewing an issue shows FleetGraph findings about it

How do you avoid redundant API calls?

- Proactive runs check `last_check_at` in config doc before fetching — if no events since last check, skip
 - WebSocket events carry enough metadata to decide whether a full graph run is needed (e.g., a title edit on a wiki doesn't trigger issue health checks)
 - Fetch nodes use conditional logic: if scope is a single issue, don't fetch all workspace issues
-

6. Human-in-the-Loop Design

Which actions require confirmation?

Any Ship API mutation: state changes, reassignment, comment creation, escalation. The line is simple: reads are autonomous, writes need approval.

What does the confirmation experience look like in Ship?

Two surfaces:

1. On-demand (chat): The agent proposes the action inline in the chat. User sees: what will happen, why, affected entities. Buttons: Confirm / Dismiss. The graph is paused at the Human Gate node until the user responds.
2. Proactive: Finding appears as a notification/card in the UI (accountability banner or a new FleetGraph findings panel). If the finding includes a proposed action, the card has Confirm / Dismiss / Snooze buttons. Snooze = suppress this specific finding for N hours.

What happens if the human dismisses or snoozes?

- Dismiss: Finding document updated to dismissed status. Agent won't re-surface the same finding for the same entity. The reasoning is preserved for audit.
 - Snooze: Finding document updated to snoozed status + `snooze_until` timestamp. Agent will re-check after the snooze expires.
 - No response (proactive): Finding stays visible but the proposed action is never executed. The agent does NOT auto-escalate on silence — that would be the agent deciding to act without approval.
-

7. Error and Failure Handling

What does the agent do when Ship API is down?

- Fetch nodes return an error state instead of crashing
- Reasoning node receives partial data + error flags
- If all fetches failed: proactive mode silently retries next cycle; on-demand mode tells the user "Ship API is unreachable, try again in a few minutes"
- If some fetches failed: reason on what we have, flag reduced confidence in the finding

How does it degrade gracefully?

- Missing team data → skip team load analysis, still do issue/sprint health
- Missing sprint data → skip scope creep detection, still do issue lifecycle checks
- LLM API down → proactive mode skips this cycle entirely; on-demand mode returns "I'm temporarily unable to analyze — here are the raw data links so you can check manually"

What gets cached and for how long?

- Team grid (changes rarely): cache 1 hour
 - Person documents: cache 1 hour
 - Project/program structure: cache 30 minutes
 - Issues/sprints: never cache (too dynamic, and the whole point is detecting changes)
 - Finding history: persisted permanently as documents in Ship
-

Phase 3: Stack and Deployment

8. Deployment Model

Where does the proactive agent run when no user is present?

FleetGraph runs as a separate lightweight service from Ship from the beginning. Ship has its own AWS Elastic Beanstalk deployment pipeline — we don't touch it. FleetGraph is a standalone process that talks to Ship's REST API over HTTP and subscribes to its /events WebSocket as a remote client.

For development and initial deployment: Linode VPS, managed by pm2. This gives us a long-lived process with automatic restart, log management, and zero coupling to Ship's AWS stack. If it needs to move to AWS later, the clean separation makes that a deployment concern, not an architecture change — just point it at the production Ship API URL.

How is it kept alive?

- pm2 manages the process (auto-restart on crash, log rotation)
- WebSocket reconnection with exponential backoff (the /events channel has 30-second ping/pong keepalive)
- The 5-minute poll acts as a heartbeat — if the poll runs, the agent is alive

How does it authenticate with Ship?

Two authentication modes:

1. Proactive mode: Uses a dedicated API token (Ship's api_tokens table). We create a service account user (e.g., fleetgraph-agent@system) and generate a long-lived Bearer token. The agent authenticates all proactive API calls with this token. This gives it read access to workspace data for monitoring.
2. On-demand mode (chat): The user's own session authenticates the request. The data flow:

```
User types in chat → Ship frontend sends message + user's session cookie
→ Ship API proxies to FleetGraph with user's auth context
→ FleetGraph fetch nodes call Ship API as that user
→ Ship returns only data that user can see
→ FleetGraph sends relevant data to OpenAI for reasoning
→ LangGraph classifies, wraps findings/actions
→ Response rendered in Ship's chat panel
```

The user's session scopes what the agent can see — it reasons about exactly what that user has access to. No privilege escalation, no shadow data.

9. Performance

How does the trigger model achieve < 5 min detection latency?

- WebSocket events arrive in <1 second

- Context resolution + parallel fetches: ~1-3 seconds (3-4 API calls)
- LLM reasoning call: ~5-15 seconds (depending on input size)
- Classification + persist: ~1-2 seconds
- Total pipeline: ~10-20 seconds from event to finding
- Poll safety net: worst case adds 5 minutes for missed WebSocket events

What is the token budget per invocation?

Estimated per reasoning call:

- Input: Context + fetched data. A typical sprint with 15-20 issues, team of 5-8 people, project metadata: ~3,000-5,000 tokens. Prompt template + system instructions: ~1,000 tokens. Total input: ~4,000-6,000 tokens.
- Output: Structured findings (severity, affected entities, reasoning, recommendation): ~500-1,500 tokens.
- Total per run: ~5,000-7,500 tokens

Using OpenAI pricing (estimated ~\$0.003/1K input, ~\$0.01/1K output):

- Input: ~\$0.012-0.018 per run
- Output: ~\$0.005-0.015 per run
- ~\$0.02-0.03 per graph run

Where are the cost cliffs?

1. Proactive runs scale with events, not users. A noisy workspace (lots of issue churn) triggers more runs. Mitigation: debounce — batch WebSocket events that arrive within a 30-second window into a single graph run.
2. On-demand scales with user engagement. Each chat message is a graph run. Mitigation: conversation context means follow-up questions can skip re-fetching (use cached state from the same session).
3. Retro pattern mining (UC7) is the most expensive use case. It reads multiple sprint retros' content — potentially 10,000+ tokens of input. Keep this on-demand only, never proactive.

Estimated cost at scale (monthly):

Assumptions: 4 proactive runs/day per workspace (debounced from events), 2 on-demand invocations/user/day, ~\$0.025 average per run.

Scale	Workspaces	Proactive/mo	On-demand/mo	Total/mo
100 users	~10	1,200 runs (\$30)	6,000 runs (\$150)	~\$180
1,000 users	~50	6,000 runs (\$150)	60,000 runs (\$1,500)	~\$1,650
10,000 users	~200	24,000 runs (\$600)	600,000 runs (\$15,000)	~\$15,600